Reducing Input Size

Case Study: Clustering (Part 3)

OUTLINE

1. k-means clustering
   - Review of popular algorithms: LLoyd's, k-means++.
   - Coreset-based approach to k-means clustering.
   - MapReduce algorithm: MR-kmeans.
   - k-median clustering (sketch)
2. Clustering evaluation
   - Unsupervised evaluation: Silhouette coefficient.
   - Other evaluation metrics (sketch).

k-means clustering

Definition (review) and some observations

Given a pointset P from a metric space (M, d) and an integer $k > 0$, the k-means clustering problem requires to determine a k-clustering $\mathcal{C} = (C_1, C_2, \ldots, C_k; c_1, c_2, \ldots, c_k)$ which minimizes:

$$\Phi_{\text{kmeans}}(\mathcal{C}) = \sum_{i=1}^{k} \sum_{a \in C_i} (d(a, c_i))^2.$$

We will first focus on:

- Euclidean spaces (i.e., $\Re^D$ with Euclidean distance).
- Unrestricted problem where centers need not belong to the input P

But we'll point out what can be done in other cases.

Observations

- k-means clustering aims at minimizing cluster variance, and works well for discovering ball-shaped clusters.
- Because of the quadratic dependence on distances, k-means clustering is rather sensitive to outliers (though less than k-center).
- Although some well-established algorithms for k-means clustering have been known for decades, only recently a rigorous assessment of their performance-accuracy tradeoff has been carried out.

Properties of Euclidean spaces

Let $X = (X_1, X_2, \ldots, X_D)$ and $Y = (Y_1, Y_2, \ldots, Y_D)$ be two points in $\Re^D$. Recall that their Euclidean distance is

$$d(X, Y) = \sqrt{\sum_{i=1}(X_i - Y_i)^2} \stackrel{\triangle}{=} ||X - Y||.$$

Definition

The centroid of a set P of N points in $\Re^D$ is

$$c(P) = \frac{1}{N} \sum_{X \in P} X,$$

mean of P

where the sum is component-wise. Note that $c(P)$ does not necessarily belong to P.

Properties of Euclidean spaces

Lemma

The centroid $c(P)$ of a set $P \subset \Re^D$ is the point of $\Re^D$ which minimizes the sum of the square distances to all points of P.

We skip the proof, although it is very simple.

Observation: The lemma implies that to minimize the kmeans objective, the best center to select for each cluster is its centroid (assuming that centers need not belong to the input pointset).

Lloyd's algorithm

- Also known as k-means algorithm, it was developed by Stuart Lloyd in 1957 and became (one of) the most popular algorithm for unsupervised learning.
- In 2006 it was listed as one of the top-10 most influential data mining algorithms.
- It relates to the Expectation-Maximization scheme since it iterates the following two steps
  - Expectation step: best clustering computed around the given centroids.
  - Maximization step: new centroids computed to minimize average square distance.

Lloyd's algorithm

Input: Set P of N points from $\Re^D$, integer $k > 1$

Output: k-clustering $\mathcal{C} = (C_1, C_2, \ldots, C_k; c_1, c_2, \ldots, c_k)$ of P with small $\Phi_{\text{kmeans}}(\mathcal{C})$. Centers need not be in P.

$\{c_1, c_2, \ldots, c_k\} \leftarrow$ arbitrary set of k points in $\Re^D$;

$\Phi \leftarrow \infty$; stopping-condition $\leftarrow$ false;

while (!stopping-condition) **do**

  for $1 \leq i \leq k$ **do**

    $C_i \leftarrow$ points of P closest to c_i;

    $c_i \leftarrow$ centroid of C_i;

  Let $\mathcal{C} = (C_1, C_2, \ldots, C_k; c_1, c_2, \ldots, c_k)$;

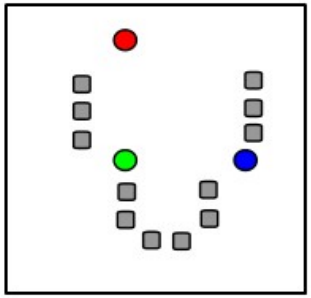
  if $(\Phi_{\text{kmeans}}(\mathcal{C}) < \Phi)$ **then** $\Phi \leftarrow \Phi_{\text{kmeans}}(\mathcal{C})$;

  else stopping-condition $\leftarrow$ true;

return $\mathcal{C}$

break ties arbitrarily

Example

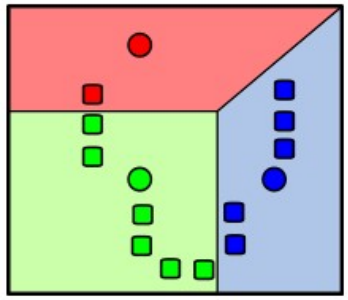


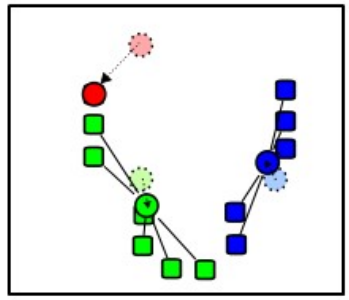
Initial centers

Partition

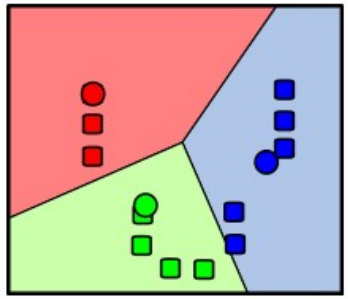
Calculation of centroids

Iteration 1:





Iteration 2:



. . .

Lloyd's algorithm: analysis

Theorem

The Lloyd's algorithm always terminates.

Proof

Let $\Phi_0 = \infty$, $\Phi_1, \Phi_2, \ldots$ be the sequence of values of variable Φ

Notice that $\Phi_i < \Phi_{i-1}$ $\forall i > 0$ except for the last iteration

Proof of Theorem

Let $\phi_i = \phi_{kmeans}(\mathcal{C}(i))$

$\mathcal{C}(i)$ clustering computed in the i-th iteration whose centers are the centroids of the clusters, thus they are uniquely defined by the clusters

$\phi_{kmeans}(\mathcal{C}(i)) < \phi_{kmeans}(\mathcal{C}(i-1))$

$\Rightarrow$ the clusters of $\mathcal{C}(i)$ and $\mathcal{C}(i-1)$ must differ, otherwise their centroids hence their

Proof of Theorem

objective functions would be the same

There are k^N possible assignments of the N points to k clusters (also counting permutations)

$\Rightarrow k^N$ is an upper bound to the number of clusterings computed by the algorithm $\Rightarrow$ an upper bound to the number of iterations $\square$

Weaknesses of Lloyd's algorithm

1. The number of iterations can be exponential in the input size.

Tighter bounds on the number of iterations are known

* $O(N^{kD})$ smaller than k^N for small k and D

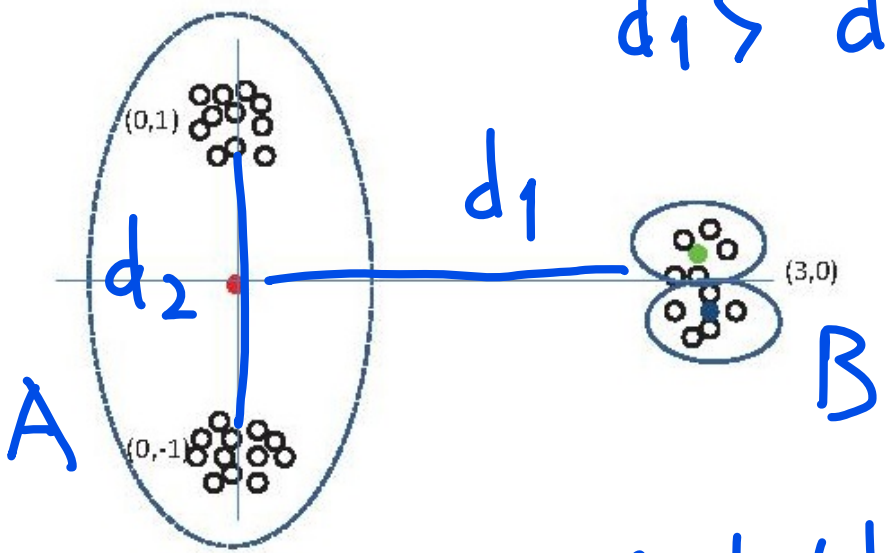
* $\geq 2^{\Omega(\sqrt{N})}$ in the worst case

Weaknesses of Lloyd's algorithm

2. The value of the objective function for the clustering returned by Lloyd's algorithm can be arbitrarily far from the optimal one since the algorithm may be trapped into a local optimum.

$k = 3$

$d_1 > d_2$



If initially 2 centers are selected from B and 1 from A, the algorithm is unable to move a center from B to A and the approx. motion factor can become $\Omega(d_2^2)$

Improving LLoyd's algorithm

1. To improve convergence:
   - Stop iterations when the improvement of the objective function drops below a fixed threshold.
   - Make a careful selection of initial centers.
2. To ensure good approximation (avoiding local minima):
   - Make a careful selection of initial centers.

Typically, initial centers are chosen at random, but there are more effective ways of selecting them.

D. Arthur and S. Vassilvitskii [AV07] developed k-means++, a careful center initialization strategy that yields clusterings with approximation guarantees.

k-means++

Let P be a set of N points from $\Re^D$, and let $k > 1$ be an integer.

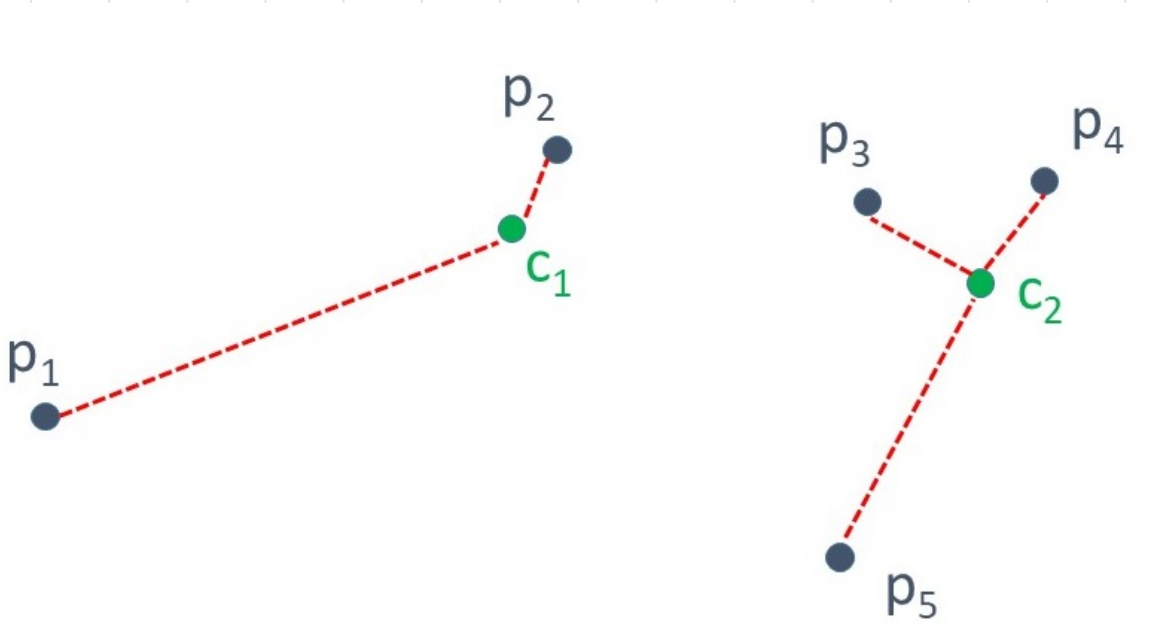
Algorithm k-means++ computes a (initial) set S of k centers for P using the following procedure (note that $S \subseteq P$):

$c_1 \leftarrow$ random point chosen from P with uniform probability;
$S \leftarrow \{c_1\}$;
for $2 \leq i \leq k$ **do**
    foreach $p \in P - S$ **do** $\pi(p) \leftarrow (d(p,S))^2 / \sum_{q \in P-S} (d(q,S))^2$;
    $c_i \leftarrow$ random point in $P - S$ according to distribution $\pi(\cdot)$;
    $S \leftarrow S \cup \{c_i\}$
return S

Probability Proportional to Distance approach

It can be implemented in $O(N\,k)$ time sequentially

k-means++



Selection of c_3

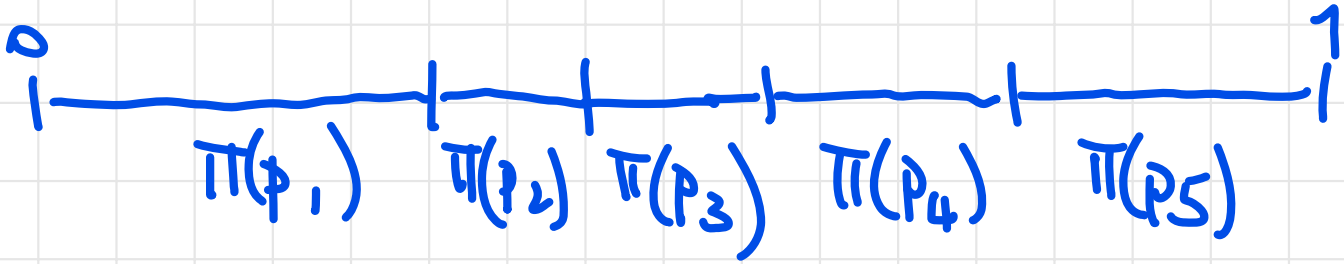
p_1 will be selected with probability

$$\Pi(p_1) \quad \frac{d(p_1,c_1)^2}{d(p_1,c_1)^2 + d(p_2,c_1)^2 + d(p_3,c_2)^2 + d(p_4,c_2)^2 + d(p_5,c_2)^2}$$

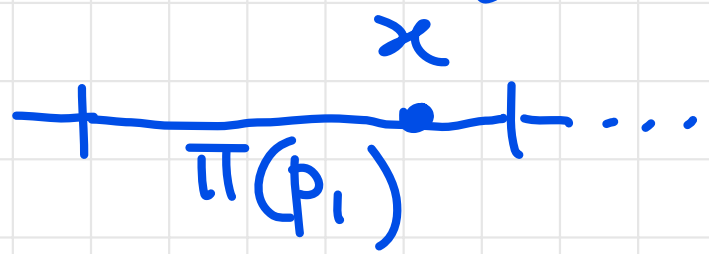
k-means++

How can we implement the selection of c_i?

Partition $[0,1]$ according to $\pi(\cdot)$



Draw a random value $x \in [0,1]$ with uniform probability



If ... then select p_1

k-means++

Let $\mathcal{C}_{\text{alg}} = \text{Assign}(P, S)$: the k-clustering of P around the set of centers S returned by k-means++.

IMPORTANT: Note that $\mathcal{C}_{\text{alg}}$ and, consequently, $\Phi_{\text{kmeans}}(\mathcal{C}_{\text{alg}})$, are random variables which depend on the random center selection!

Theorem (Arthur-Vassilvitskii'07)

$$E[\Phi_{\text{kmeans}}(\mathcal{C}_{\text{alg}})] \leq 8(\ln k + 2) \cdot \Phi^{\text{opt}}_{\text{kmeans}}(P, k).$$

N.B. We skip the proof which can be found in the original paper.

k-means++ is able to provide an $8(\ln k + 2)$-approx to kmeans clustering problem

Observations

* k-means++ exhibits the same accuracy even for other metric spaces and for the k-median clustering problem
* the centers returned by k-means++ belong to P
* If $k' = \beta \cdot k$ centers are selected by k-means++ with $\beta > 1$, then the clustering induced by these centers has an objective function close to $\Phi^{opt}_{kmeans}(P, k)$ (bi-criteria approximation)

k-means clustering for big data

How can we compute a "good" k-means clustering of a pointset P that is too large for a single machine?

Main alternatives:

- Distributed (e.g., MapReduce) implementation of Lloyd's algorithm and k-means++.

  (The next two exercises explore this alternative.)

- Coreset-based approach, similar in spirit to what was done for k-center. This approach lends itself to efficient distributed (e.g., MapReduce) as well as streaming implementations.

Lloyd's and k-means++ in MapReduce

Do the following two exercises assuming that the algorithms are applied to a set P of N points from $\Re^D$, with D constant, and that the target number of clusters is $k \in (1, \sqrt{N}]$.

Exercise

Show how to implement Lloyd's algorithm in MapReduce using $O(1)$ rounds per iteration, local space $M_L = O\left(\sqrt{N}\right)$ and aggregate space $M_A = O(N)$. (**Hint:** Make the set of centers and Φ global variables.)

Exercise

Show how to implement algorithm k-means++ in MapReduce using $O(k)$ rounds, local space $M_L = O\left(\sqrt{N}\right)$ and aggregate space $M_A = O(N)$.

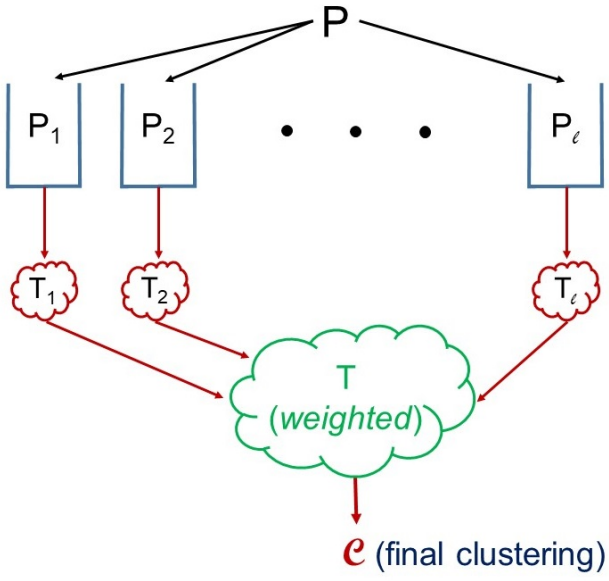
Observations

* If we use a distributed implementation of Lloyd's and/or k-means++, the number of rounds may be large
* There is a parallel variant of k-means++ dubbed k-means|| (see [B+12]) using
  - $O(\log N)$ rounds to select a coreset of $> k$ points
  - weighted implementation of k-means++ to select the final centers

k-means|| is used in the Spark mllib library as possible initialization for Lloyd's algorithm

Coreset-based approach for k-means

We use a coreset-based approach similar as for k-center.

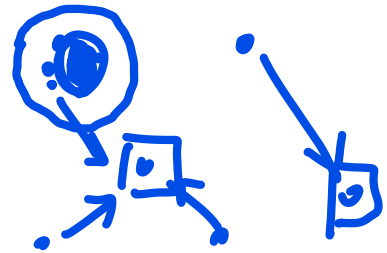


- The local coresets T_i's and the final clustering $\mathcal{C}$ are computed using a sequential algorithm for k-means.
- Each point of T is given a weight equal to the number of points in P it represents, so to account for their cumulative contribution to the objective function

Need a weighted variant of k-means clustering.

Weighted k-means clustering

Weighted variant of the k-means clustering problem



Input:

- Set P of N points from a metric space (M, d).
- Weight $w(p) > 0$ for every $p \in P$.
- Target number k of clusters.

Output: k-clustering $\mathcal{C} = (C_1, C_2, \ldots, C_k; c_1, c_2, \ldots, c_k)$ of P which minimizes the following objective function

$$\Phi^{w}_{\text{kmeans}}(\mathcal{C}) = \sum_{i=1}^{k} \sum_{p \in C_i} w(p) \cdot (d(p, c_i))^2,$$

Weighted k-means clustering: observations

- The unweighted k-means clustering problem is a special case of the weighted variant where all weights are equal to 1.
- Most known algorithms can be adapted to solve the weighted variant.
- For integer weights, it is sufficient to regard each point p as representing $w(p)$ identical copies of itself. The same approximation guarantees are maintained.

Lloyd's algorithm with weights

The only modification to the algorithm which is needed regards

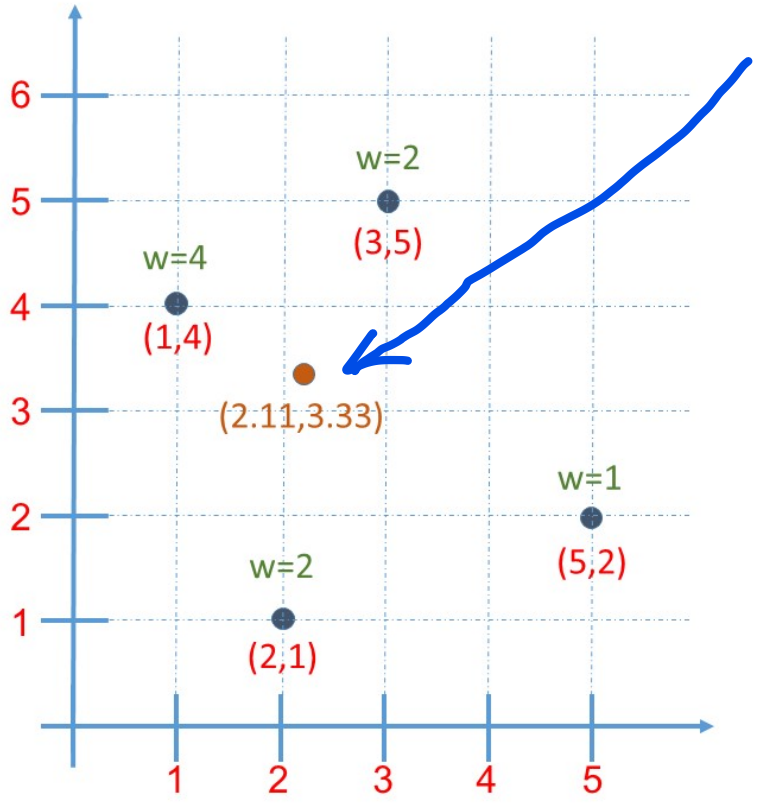
* Computation of centroid for a cluster C_i

$$c_i = \frac{1}{\sum_{p \in C_i} w(p)} \cdot \sum_{p \in C_i} w(p) \cdot p$$

each coordinate of p is multiplied by $w(p)$

* $\phi_{kmeans}(\mathcal{C}) \longrightarrow \phi^{w}_{kmeans}(\mathcal{C})$

Example of weighted centroid



$$\text{centroid} = \frac{4\cdot(1,4)+2(3,5)+2(2,1)+1(5,2)}{9}$$

$$= \frac{(4+6+4+5,\ 16+10+2+2)}{9}$$

$$= \left(\frac{19}{9},\ \frac{30}{9}\right) \simeq (2.11,\ 3.33)$$

k-means++ algorithm with weights

The only modification to the algorithm which is needed regards the computation of the probabilities for center selection

S = current set of centers

The next center selected from $P-S$ is p with probability

$$\pi(p) = \frac{w(p) \cdot (d(p,S))^2}{\sum_{q \in P-S} w(q) \cdot (d(q,S))^2}$$

Coreset-based MapReduce algorithm for k-means

MR-kmeans($\mathcal{A}$): MapReduce algorithm for k-means (described in the next slide) which uses a sequential algorithm $\mathcal{A}$ for k-means, as a parameter.

In principle, one can instantiate $\mathcal{A}$ with any sequential algorithm, as long as the following characteristics are met:

- $\mathcal{A}$ solves the more general weighted variant.
- $\mathcal{A}$ requires space proportional to the input size (if larger space is needed, the analysis of M_L for the MapReduce algorithm must be amended).

MR-kmeans($\mathcal{A}$)

Input Set P of N points from a metric space (M, d), integer $k > 1$

Output k-clustering $\mathcal{C} = (C_1, C_2, \ldots, C_k; c_1, c_2, \ldots, c_k)$ of P which is a good approximation to the k-means problem for P.

Algorithm MR-kmeans($\mathcal{A}$):

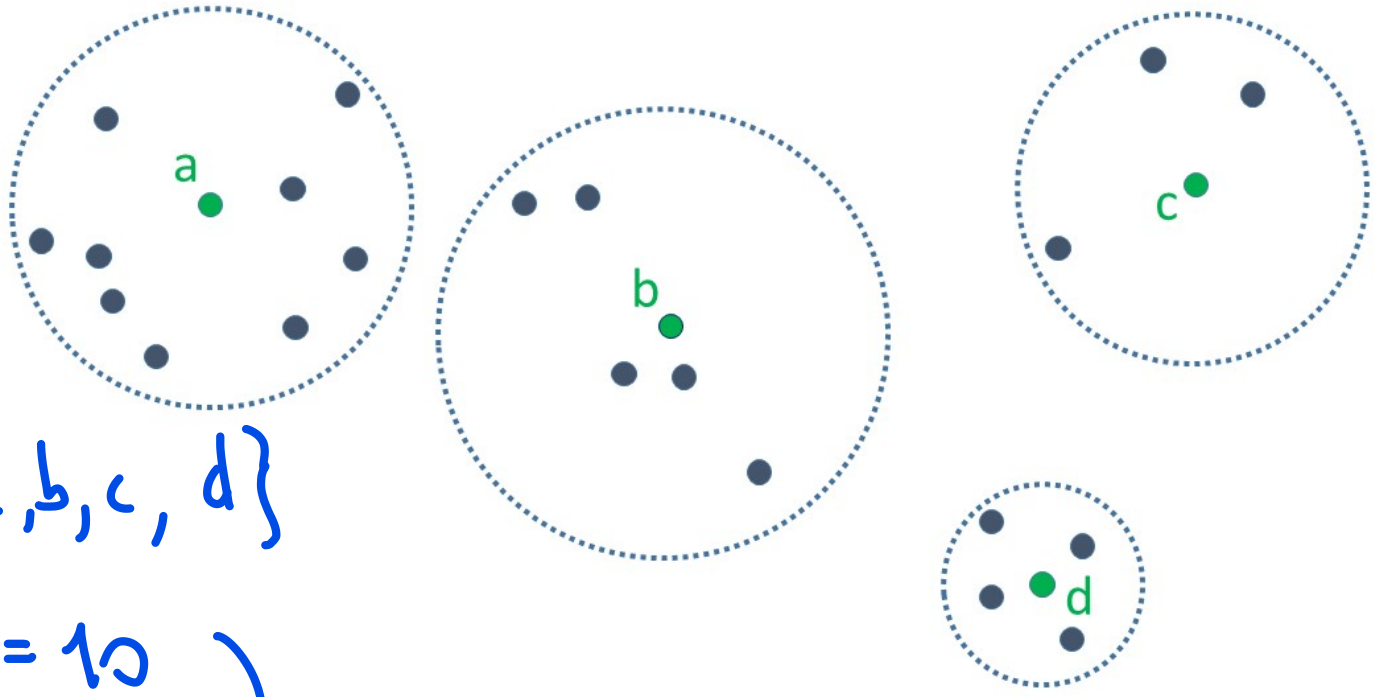
- **Round 1:**
  - Map Phase: Partition P arbitrarily in ℓ subsets of equal size $P_1, P_2, \ldots, P_\ell$.
  - Reduce Phase: for every $i \in [1, \ell]$ separately, run $\mathcal{A}$ on P_i (with unit weights) to determine a set $T_i \subseteq P_i$ of k centers, and define
    - For each $p \in P_i$, the proxy $\pi(p)$ as p's closest center in T_i.
    - For each $q \in T_i$, the weight $w(q)$ as the number of points of P_i whose proxy is q.

MR-kmeans($\mathcal{A}$)

- **Round 2:**
  - Map Phase: empty.
  - Reduce Phase: gather the coreset $T = \cup_{i=1}^{\ell} T_i$ of $\ell \cdot k$ points, together with their weights, and run, using a single reducer, $\mathcal{A}$ on T (with the given weights) to determine a set $S = \{c_1, c_2, \ldots, c_k\}$ of k centers.
- **Round 3:** Execute Assign(P, S).

Example for Round 1

Set T_i (green points) computed in a partition P_i



$T_i = \{a, b, c, d\}$

$w(a) = 10$

$w(b) = 6$

$w(c) = 4$

$w(d) = 5$

} assumes $T_i \subseteq P_i$ otherwise you subtract 1 from the weights

Analysis of MR-kmeans($\mathcal{A}$)

Assume $k \leq \sqrt{N}$. By setting $\ell = \sqrt{N/k}$, it is easy to see that MR-kmeans($\mathcal{A}$) algorithm can be implemented using

- Local space $M_L = O\left(\max\{N/\ell, \ell \cdot k\}\right) = O\left(\sqrt{N \cdot k}\right) = o(N)$
- Aggregate space $M_A = O(N)$

Same analysis as MR-F.F.T.

The quality of the solution returned by the algorithm is stated in the following theorem, which is proved in the next few slides.

Theorem

Suppose that $\mathcal{A}$ is an α-approximation algorithm for weighted k-means clustering, for some $\alpha > 1$, and let $\mathcal{C}_{\text{alg}}$ be the k-clustering returned by MR-kmeans($\mathcal{A}$) with input P. Then:

$$\Phi_{\text{kmeans}}(\mathcal{C}_{\text{alg}}) = O\left(\alpha^2 \cdot \Phi^{\text{opt}}_{\text{kmeans}}(P, k)\right).$$

That is, MR-kmeans($\mathcal{A}$) is an $O\left(\alpha^2\right)$-approximation algorithm.

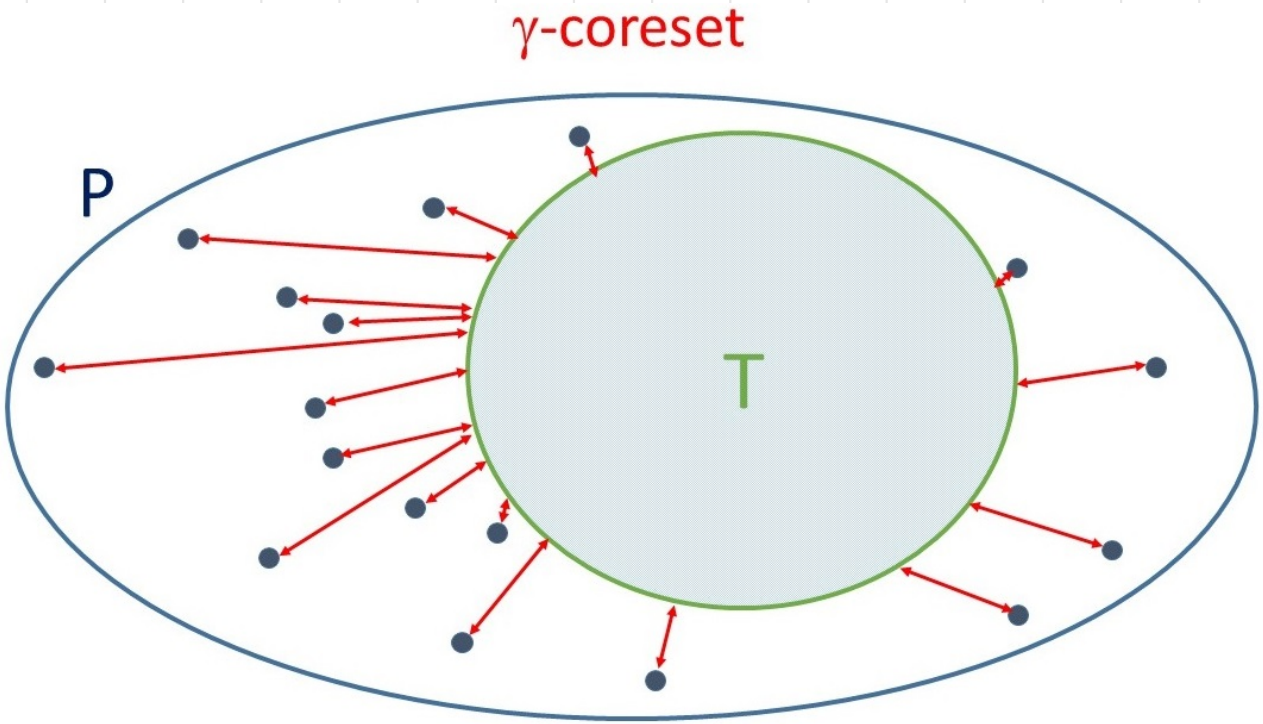
Sequential α-approx. $\longrightarrow$ MapReduce α^2-approximation

Analysis of MR-kmeans($\mathcal{A}$)

Definition

Given a pointset P, a coreset $T \subseteq P$ and a proxy function $\pi : P \to T$, we say that T is a γ-coreset for P, k and the k-means objective if

$$\sum_{p \in P} (d(p, \pi(p)))^2 \leq \gamma \cdot \Phi^{\text{opt}}_{\text{kmeans}}(P, k).$$



T is a γ-coreset if the sum of squared distances from P-T to T (red lines) is

$\leq \gamma \cdot \phi^{OPT}_{kmeans}(P,k)$

The smaller γ and the better T represents P

Proof of Theorem (sketch)

Lemma 1 (we skip proof)

* if $A \equiv$ α-approximation algorithm for weighted k-means clustering

* if T (coreset gathered in Round 2) is a γ coreset

$$\Rightarrow \phi_{kmeans}(C_{alg}) = O\left(\alpha(1+\gamma)\phi^{OPT}_{kmeans}(P,k)\right)$$

Proof of Theorem (sketch)

Obs. factor α comes from computation of the solution on T

factor $(1+\gamma)$ comes from restricting the search for the solution (i.e. the search of the centers) on T rather than on the entire P

Lemma 2 If an α-approximation alg. A is used to compute the T_i's then $T = \bigcup_{i=1}^{l} T_i$ is a γ-coreset with $\gamma = \alpha$

Proof of Theorem (sketch)

(we skip the proof)

Proof of theorem is an immediate consequence of the 2 lemmas.

Proof of Theorem (sketch)

Observations

- Two different k-means sequential algorithms can be used in R1 and R2. For example, one could use k-means++ in R1, and k-means++ plus LLoyd's in R2.
- In practice, the algorithm is fast and accurate.
- If $k' > k$ centers are selected from each P_i in R1 (e.g., through k-means++), the quality of T, hence the quality of the final clustering, improves.

k-median clustering (sketch)

Given a pointset P from a metric space (M, d) and an integer $k > 0$, the k-median clustering problem requires to determine a k-clustering $\mathcal{C} = (C_1, C_2, \ldots, C_k; c_1, c_2, \ldots, c_k)$ which minimizes:

$$\Phi_{\text{kmedian}}(\mathcal{C}) = \sum_{i=1}^{k} \sum_{a \in C_i} d(a, c_i).$$

- $O(1)$ sequential approximation algorithms for k-median are known but the most accurate ones are computationally intensive.
- The most popular sequential algorithm, PAM: Partitioning Around Medoids (a.k.a. k-medoids algorithm) is based on a local search optimization strategy.
- A coreset-based approach similar to MR-kmeans can be employed where the use of accurate sequential strategies is confined to a coreset much smaller than the input.

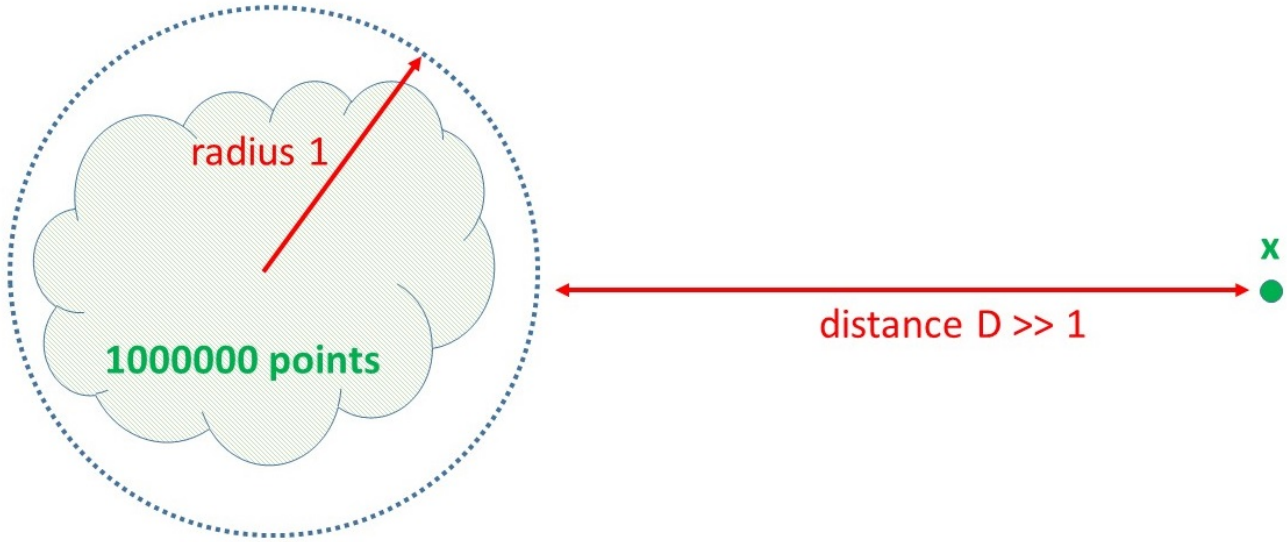
k-center vs k-means vs k-median

Some final considerations on the 3 center-based clustering problems.

- All 3 problems tend to return spherically-shaped clusters.
- They differ in the center selection strategy since, given the centers, the best clustering around them is the same for the 3 problems.
- Suitable coreset-based strategies can be employed for the 3 problems, featuring:
  - Ability to handle huge inputs.
  - Tunable performance-accuracy tradeoffs.
  - Very good behaviour in practice.
- The selection of the best k is often difficult, and this is a drawback common to the 3 problems.

k-center vs k-means vs k-median

The following example shows that different impact that outliers can have for the 3 problems.



For $k \geq 2$, the relevance of the outlier x in the 3 problems is different

- **k-center:** x must be always selected as center!
- **k-means:** x must be selected as center if $D \geq 1000$!
- **k-median:** x must be selected as center if $D \geq 1000000$!

Exercises

Exercise

Let $\mathcal{C}_{\text{alg}}$ be the k-clustering of P induced by the set S of centers returned by k-means++ (i.e., $\mathcal{C}_{\text{alg}} = \text{Assign}(P, S)$). Using Markov's inequality determine a value $\alpha > 1$ such that

$$\Pr\left(\Phi_{\text{kmeans}}(\mathcal{C}_{\text{alg}}) \leq \alpha \cdot \Phi^{\text{opt}}_{\text{kmeans}}(P, k)\right) \geq 1/2.$$

Recall that for a real-valued nonnegative random variable X with expectation $E[X]$ and a value $a > 0$, the Markov's inequality states that

$$\Pr(X \geq a) \leq \frac{E[X]}{a}.$$

Exercises

Exercise

(*This exercise shows how to make the approximation bound of algorithm k-means++ to hold with high probability rather than in expectation.*)

Let $\mathcal{C}_{\text{alg}}$ be the k-clustering of P induced by the set S of centers returned by k-means++ (i.e., $\mathcal{C}_{\text{alg}} = \text{Assign}(P, S)$). Suppose that we know that for some value $\alpha > 1$ the following probability bound holds:

$$\Pr\left(\Phi_{\text{kmeans}}(\mathcal{C}_{\text{alg}}) \leq \alpha \cdot \Phi^{\text{opt}}_{\text{kmeans}}(P, k)\right) \geq 1/2.$$

Show that the same approximation ratio, but with probability at least $1 - 1/N$, can be ensured by running several *independent instances* of k-means++, computing for each instance the clustering around the returned centers, and returning at the end the best clustering found (call it $\mathcal{C}_{\text{best}}$), i.e., the one with smallest value of the objective function.

Clustering evaluation

Unsupervised Evaluation

- Let P be a set of N points from a metric space (M, d)
- Let $\mathcal{C} = (C_1, C_2, \ldots, C_k)$ be a partition of P into k clusters (i.e., a k-clustering ignoring cluster centers, if available).

Goal of unsupervised evaluation: assess the *quality* of $\mathcal{C}$ without reference to external information or ground truth.

Quality measures:

- Specific objective function, *if any*, which was used to select $\mathcal{C}$ among all feasible clusterings. For example k-center/k-means/k-median objective functions which, however, capture only intra-cluster similarity.
- Silhouette coefficient, which captures both intra-cluster similarity and inter-class dissimilarity.

cluster

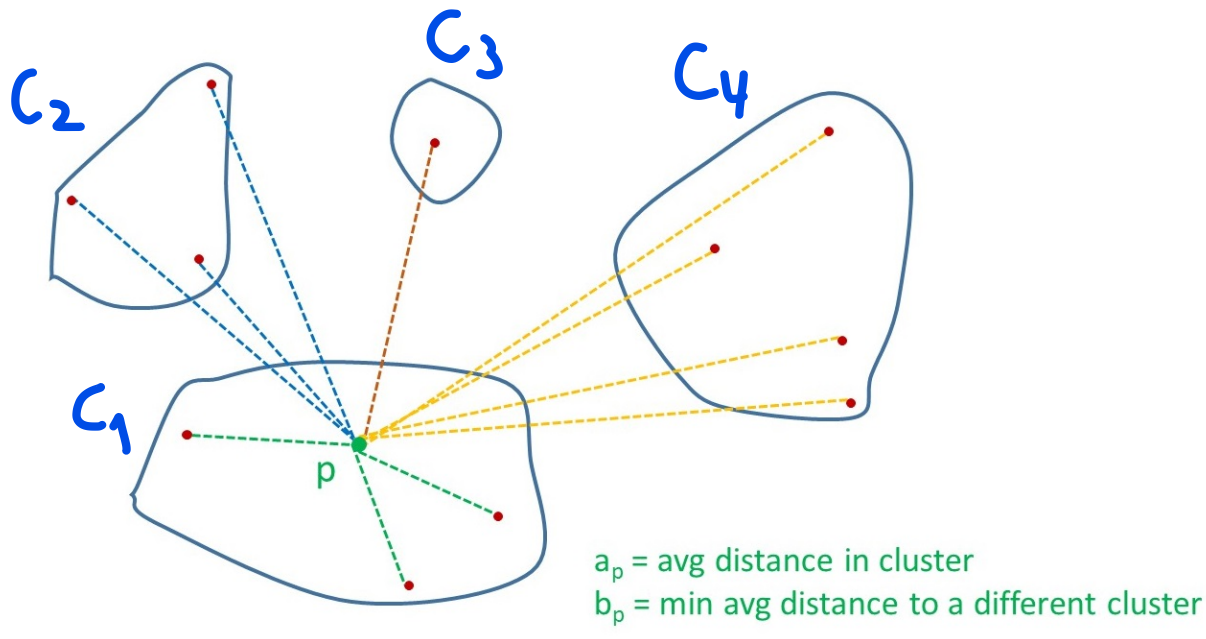
Observation: the silhouette coefficient is often used to select the most suitable number of clusters k, for a given clustering algorithm.

Unsupervised evaluation: silhouette

For a point $p \in P$ belonging to some cluster C_i let

$$\begin{aligned} a_p &= d_{\text{sum}}(p, C_i)/|C_i| \\ b_p &= \min_{j \neq i} d_{\text{sum}}(p, C_j)/|C_j|, \end{aligned}$$

where $d_{\text{sum}}(p, C)$ denotes the sum of the distances between p and the points of a cluster C (i.e., $d_{\text{sum}}(p, C) = \sum_{q \in C} d(p, q)$).



Unsupervised evaluation: silhouette

Definition: silhouette coefficient

Let $\mathcal{C} = (C_1, C_2, \ldots, C_k)$ be a k-clustering of P (ignoring possible centers). For any $p \in P$ the silhouette coefficient for p is

$$s_p = \frac{b_p - a_p}{\max\{a_p, b_p\}},$$

assess whether p is clustered well

To measure the quality of $\mathcal{C}$ we define the average silhouette coefficient as

$$s_{\mathcal{C}} = \frac{1}{|P|} \sum_{p \in P} s_p.$$

Unsupervised evaluation: silhouette

$|b_p - a_p| \leq \max\{a_p, b_p\}$

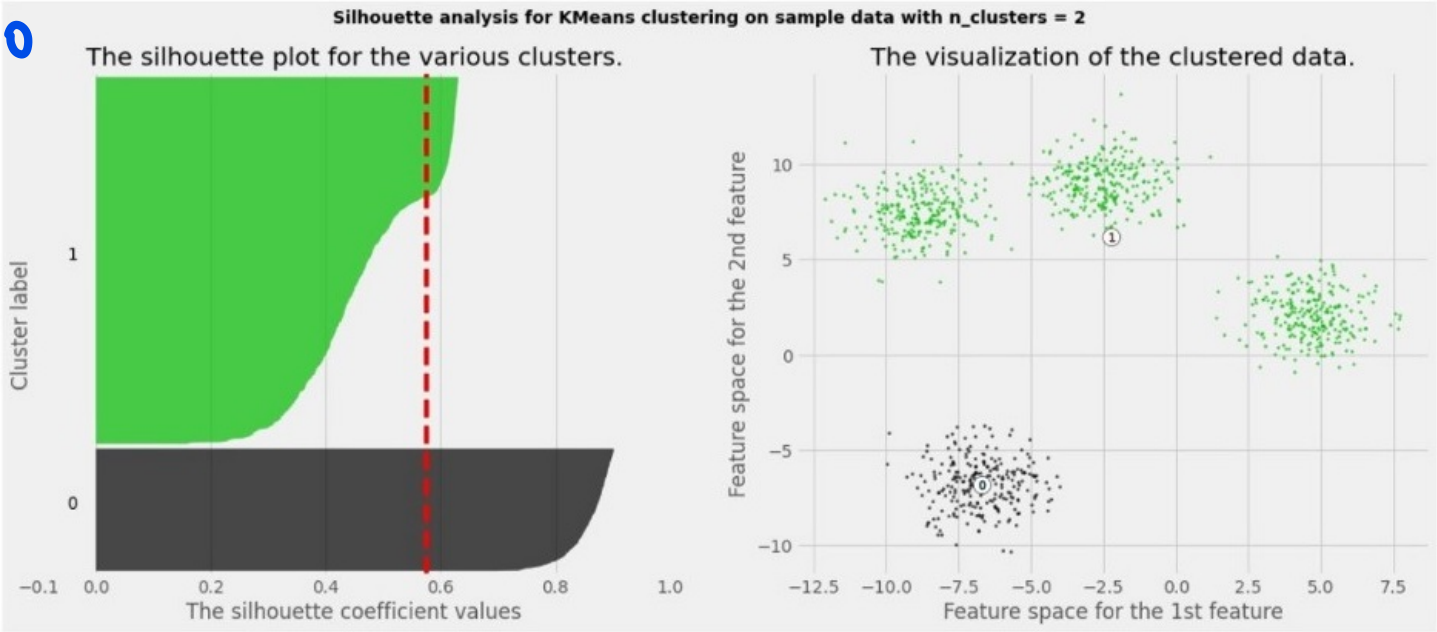
$\Rightarrow s_p \in [-1, 1] \Rightarrow s_C \in [-1, 1]$

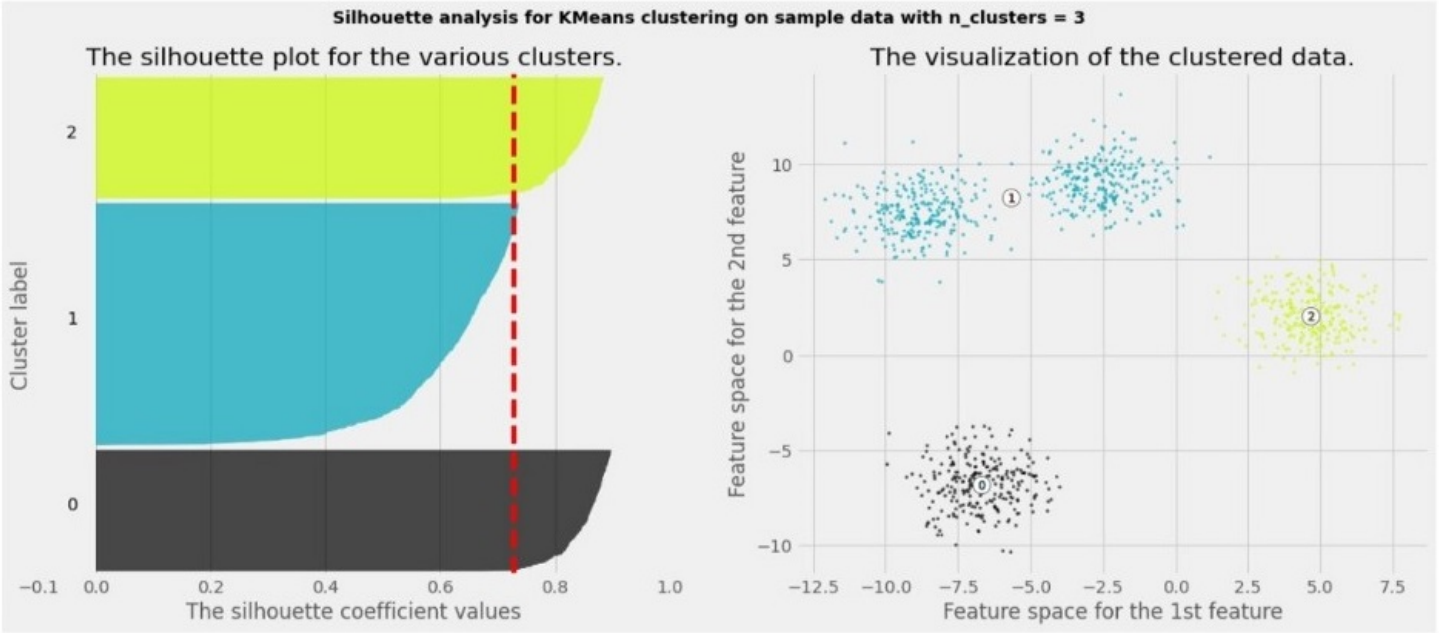
$s_p \approx -1$ if $a_p \gg b_p$ i.e., p is NOT CLUSTERED WELL

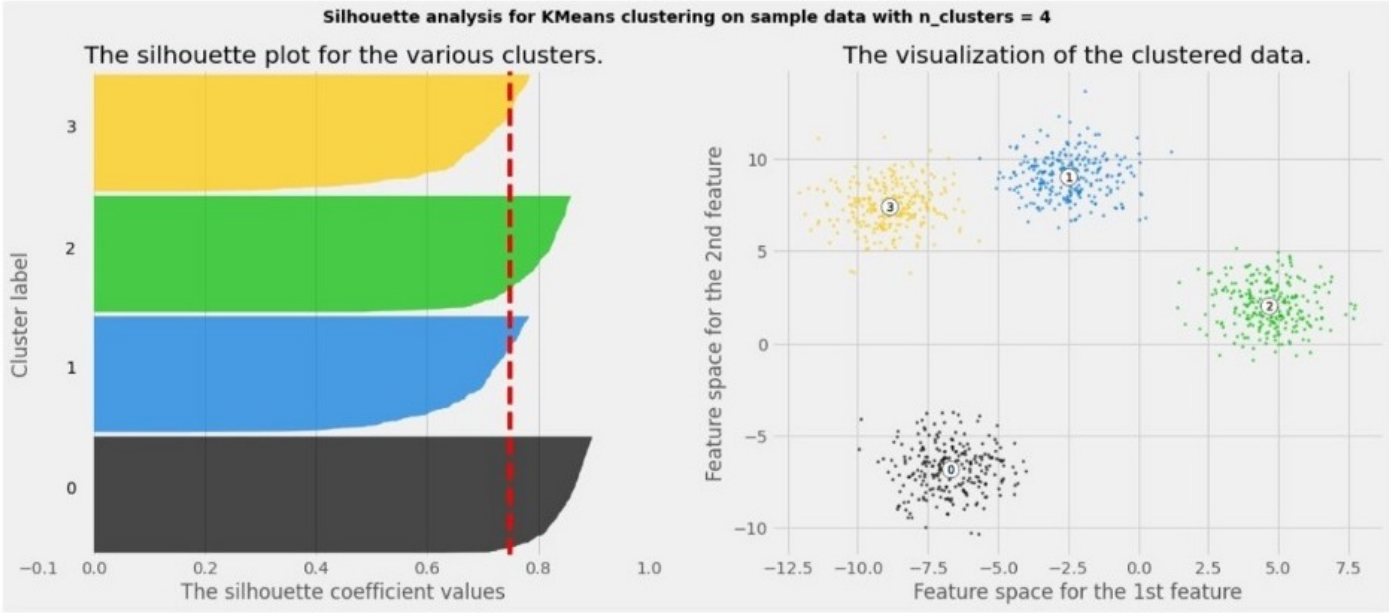
$s_p \approx 1$ if $a_p \ll b_p$ i.e., p is CLUSTERED WELL

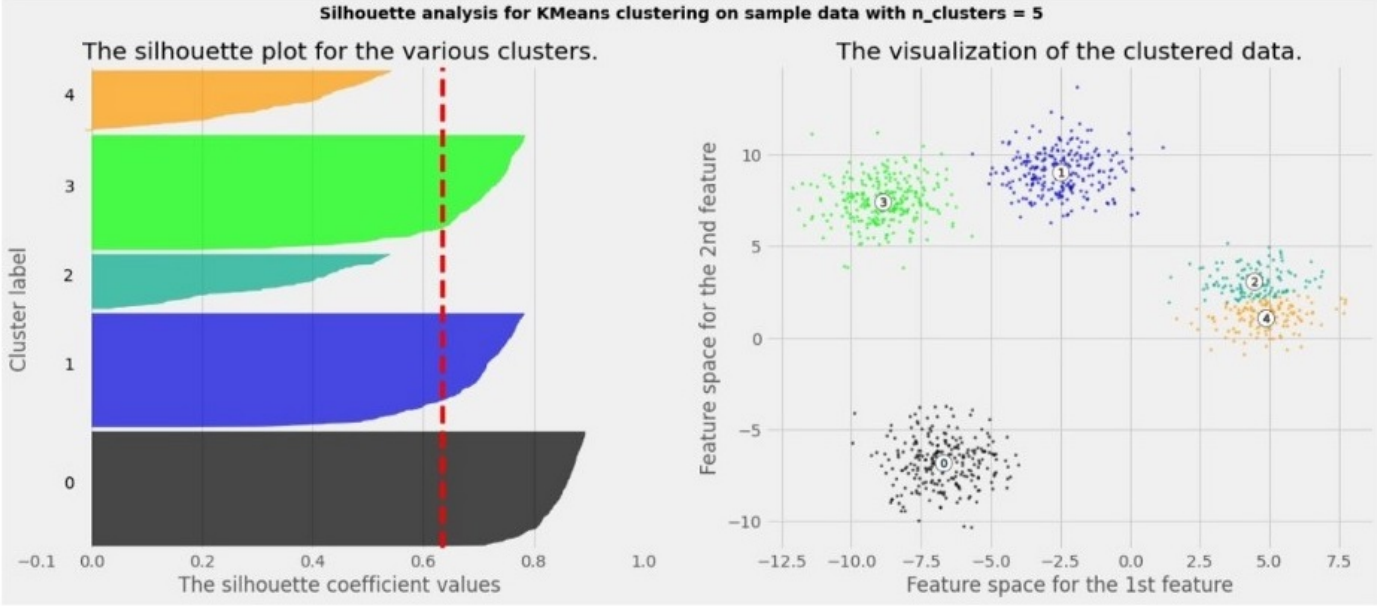
C is GOOD if s_C is close to 1 (i.e., most points are clustered well)

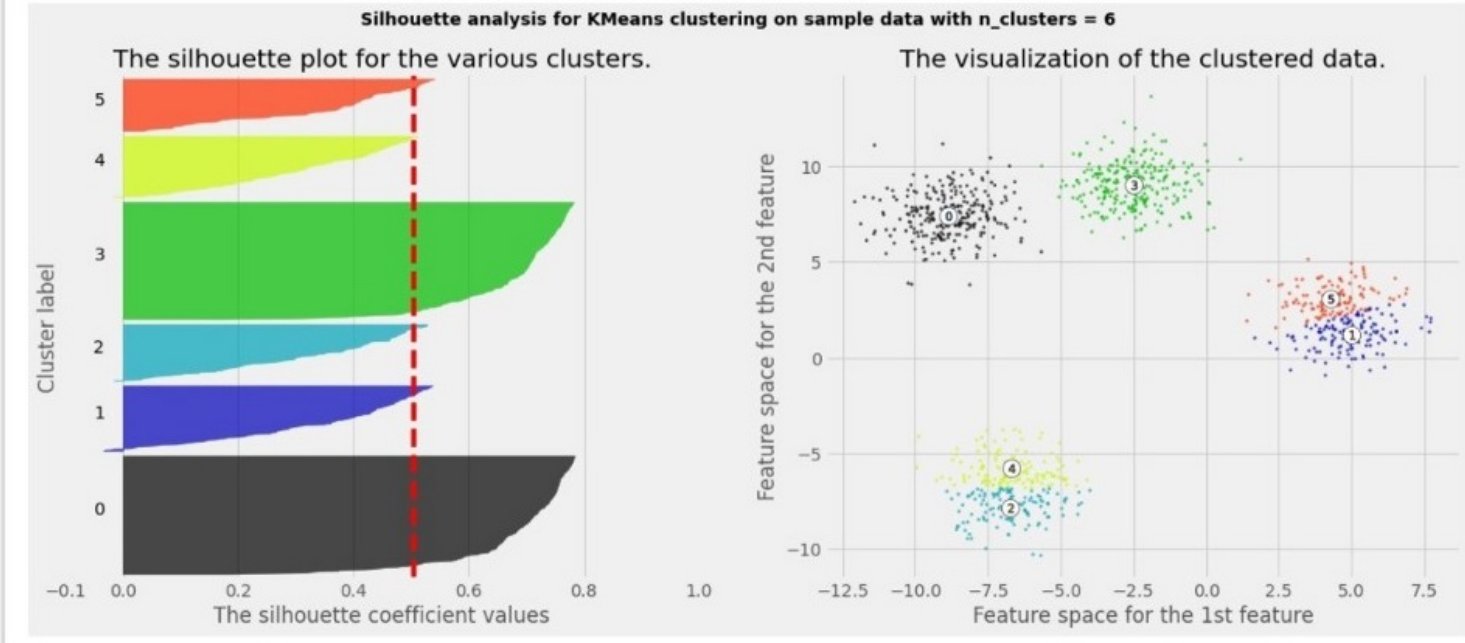
$|P| = 500$


Silhouette analysis for KMeans clustering on sample data with n_clusters = 2
The silhouette plot for the various clusters.
The visualization of the clustered data.
Cluster label
The silhouette coefficient values
Feature space for the 2nd feature
Feature space for the 1st feature


Silhouette analysis for KMeans clustering on sample data with n_clusters = 3
The silhouette plot for the various clusters.
The visualization of the clustered data.
Cluster label
The silhouette coefficient values
Feature space for the 2nd feature
Feature space for the 1st feature


Silhouette analysis for KMeans clustering on sample data with n_clusters = 4
The silhouette plot for the various clusters.
Cluster label
The silhouette coefficient values
The visualization of the clustered data.
Feature space for the 2nd feature
Feature space for the 1st feature


Silhouette analysis for KMeans clustering on sample data with n_clusters = 5
The silhouette plot for the various clusters.
Cluster label
The silhouette coefficient values
The visualization of the clustered data.
Feature space for the 2nd feature
Feature space for the 1st feature


Silhouette analysis for KMeans clustering on sample data with n_clusters = 6
The silhouette plot for the various clusters.
Cluster label
The silhouette coefficient values
The visualization of the clustered data.
Feature space for the 2nd feature
Feature space for the 1st feature

Computational considerations

- The straightforward computation of $s_{\mathcal{C}}$ requires $\Theta\left(|P|^2\right)$ distance computations, unfeasible for very large $|P|$.
- Simplified computation for cosine and squared Euclidean distances ($\Theta\left(|P|k\right)$ distance computations). Methods are provided in the Spark library.
- There exhist heuristics to approximate $s_{\mathcal{C}}$, some with provable guarantees (from Padova!).

Approximating the sum of distances

Consider a point p and a set C. The following method can be used to approximate $d_{\text{sum}}(p, C)$. Let $n = |C|$.

1. Fix a target sample size $t \in [1, n]$.
2. Select a sample $S_t \subset C$, where each $x \in C$ is independently included in S_t with probability t/n (*Poisson sampling*)..
3. Compute the approximation

$$\tilde{d}_{\text{sum}}(p, C, t) = \frac{n}{t} \sum_{x \in S_t} d(p, x)$$

$\Rightarrow E[|S_t|] = t$

$= \sum_{x \in C} V(x)$

Remark. $\tilde{d}_{\text{sum}}(p, C, t)$ *is a random variable, whose value depends on the random sample* S_t.

Proposition

$\tilde{d}_{\text{sum}}(p, C, t)$ is an unbiased estimator of $d_{\text{sum}}(p, C)$, i.e.,

$$E[\tilde{d}_{\text{sum}}(p, C, t)] = d_{\text{sum}}(p, C).$$

Proof of Proposition

$\forall x \in C$ random variable $V(x)$

$$V(x) = \begin{cases} d(p,x) & \text{with probability } \frac{t}{n} \\ 0 & \text{with probability } 1 - \frac{t}{n} \end{cases}$$

$$\Rightarrow \tilde{d}_{sum}(p,C,t) = \frac{n}{t} \sum_{x \in C} V(x)$$

$$\Rightarrow E[\tilde{d}_{sum}(p,C,t)] = \frac{n}{t} \sum_{x \in C} E[V(x)]$$ by linearity of expectation

Proof of Proposition

$$\frac{n}{t} \sum_{x \in C} E[V(x)] = \frac{n}{t} \sum_{x \in C} \frac{t}{n} \cdot d(p, x)$$

$$= \sum_{x \in C} d(p, x)$$

$$= d_{sum}(p, C)$$

□

Approximating the sum of distances

Observations:

- The absolute error of the estimate $|\tilde{d}_{\text{sum}}(p, C, t) - d_{\text{sum}}(p, C)|$ can be upper bounded analytically as a function of t (see [EW04]).
- While the theoretical bound is somewhat weak, in practice the error is rather low even with small t.

Exercise

Let P be a pointset of N points from a metric space (M, d) and let $C \subseteq P$ be a subset of size n. Show how to compute $\tilde{d}_{\text{sum}}(p, C, t)$ efficiently in MapReduce, for all $p \in P$. Assume that t and n are know and

- $t \in O(\log N)$ but n can be large (up to N);
- each point $x \in P$ is represented by a pair $(ID_x, (x, f_x))$, where ID_x is a distinct integer in $[0, N-1]$ and f_x is a binary flag with value 1 if $x \in C$, andn 0 otherwise.

Approximating the Silhouette

Let $\mathcal{C} = (C_1, C_2, \ldots, C_k)$ be a k-clustering of P.

Fix a sample size t and define $t_i = \min\{t, |C_i|\}$, for $1 \leq i \leq k$.

For $1 \leq i \leq k$ and $p \in C_i$, define

$$\tilde{a}_p = \tilde{d}_{\text{sum}}(p, C_i, t_i)/|C_i| \quad \text{and} \quad \tilde{b}_p = \min_{j \neq i} \tilde{d}_{\text{sum}}(p, C_j, t_j)/|C_j|,$$

The approximate silhouette coefficient of p is

$$\tilde{s}_p = \frac{\tilde{b}_p - \tilde{a}_p}{\max\{\tilde{a}_p, \tilde{b}_p\}},$$

The approximate average silhouette coefficient of $\mathcal{C}$ is

$$\tilde{s}_{\mathcal{C}} = (1/|P|) \sum_{p \in P} \tilde{s}_p.$$

Exercise

Determine the expected number of distance computations needed to obtain $\tilde{s}_{\mathcal{C}}$.

Assume that only one sample from each cluster C_i is used for computing the $\tilde{d}_{sum}(p, C_i, t_i)$'s for all p's. Let S_i denote the sample extracted from C_i and note that

$$E[|S_i|] = t_i \leq \min\{t, |C_i|\}$$

The expected number of distance computations is

$$|P| \cdot \sum_{i=1}^{k} E[|S_i|] = |P| \cdot \sum_{i=1}^{k} t_i \leq |P| \cdot t \cdot k$$

Other evaluation metrics

- Supervised evaluation: assessment of the quality of a clustering with reference to external information. For example, Entropy can be used to measure the coherence of a clustering with respect to a ground truth (e.g., class labes).
- Clustering tendency: assessment whether the data contain meaningful clusters, namely clusters that are unlikely to occur in random data. The Hopkins statistic can be used to this purpose.

Clustering evaluation is well described in [TSK06]

Summary of Parts 1 and 2

- Metric space and prominent distance functions.
- Combinatorial optimization problem and approximation algorithm.
- k-center clustering:
  - Definition of the problem.
  - Farthest-First Traversal algorithm.
  - (Composable) coreset technique.
  - MR-Farthest-First Traversal.
- k-means clustering:
  - Definition of the problem.
  - Review of Lloyd's and k-means++ algorithms.
  - Weighted variant of the problem.
  - MR-kmeans
  - k-median clustering (sketch).

Summary of Parts 1 and 2

- Unsupervised clustering evaluation:
  - Silhouette coefficient.
  - Approximating the sum of distances.
  - Approximating the Silhouette.
- Other evaluation metrics (sketch).

References

LRU14 J. Leskovec, A. Rajaraman and J. Ullman. Mining Massive Datasets. Cambridge University Press, 2014. Sections 3.1.1 and 3.5, and Chapter 7

BHK18 A. Blum, J. Hopcroft, and R. Kannan. Foundations of Data Science. Manuscript, June 2018. Chapter 7

AV07 D. Arthur, S. Vassilvitskii. k-means++: the advantages of careful seeding. Proc. of ACM-SIAM SODA 2007: 1027-1035.

B+12 B. Bahmani et al. Scalable K-Means++. PVLDB 5(7):622-633, 2012

C+19 V. Cohen-Addad et al. Tight FPT Approximations for k-Median and k-Means. Proc. of ICALP 2019: 42:1-42:14.

Wei16 D. Wei A Constant-Factor Bi-Criteria Approximation Guarantee for k-means++. Proc. of NIPS 2016: 604-612.

TSK06 P.N.Tan, M.Steinbach, V.Kumar. Introduction to Data Mining. Addison Wesley, 2006. Chapter 8.

EW04 D. Eppstein, J. Wang: Fast Approximation of Centrality. J. Graph Algorithms Appl. 8: 39-45 (2004).